

Custom Container Images



Create Images with Containerfiles

- Docker uses Dockerfile, Podman uses Containerfile
- Consists of instructions that runtime uses to build container image.
- Image can be used to start any number of containers.
- Each instruction captured in resulting image layer.
- All layers stacked together to form a read-only container image.

Choosing a Base Image

When you build an image, podman executes the instructions in the Containerfile and applies the changes on top of a container *base image*. A base image is the image from which your Containerfile and its resulting image is built.

The base image you choose determines the Linux distribution and its components, such as:

- Package manager
- Init system
- Filesystem layout
- Preinstalled dependencies and runtimes

The base image can also influence other factors, such as image size, vendor support, and processor compatibility.

Red Hat Universal Base Images (UBI)

- Intended as common starting point for containers.
- Supported, curated and regularly updated by Red Hat team.
- Comes in four variants:
 1. **Micro:** Smallest UBI. Includes bare minimum number of packages. Does not have package manager.
 2. **Minimal:** Smaller than init. Still provides nice-to-have features such as microdnf.
 3. **Init:** Simplifies running multiple applications within single container. Has all features of the minimal UBI plus systemd.
 4. **Standard:** Primary UBI. Includes full-sized DNF package manager, systemd, gzip and tar.

Containerfile Instructions

- Uses domain-specific language (DSL).
- Mainly basic instructions to craft out container images.
- Most commonly used instructions:

FROM: set base image.

WORKDIR: set current working directory within container.

COPY: Copy file from builder host into container image. Use WORKDIR for relativity

ADD: Copying files from URLs and able to Unpack tar archives. Behaves like COPY if no URLs or tar is used.

RUN: Execute linux command within container. Always results in new layer.

ENTRYPOINT: Set final executable to run when container is started. Often defined as last instruction.

CMD: Runs command when container is started. When combined with ENTRYPOINT, CMD passes as argument.

USER: Changes active user within container. Instructions follow USER instruction run as this user, including CMD and ENTRYPOINT instruction. Good practice to define non-root for security reasons

LABEL: Add key-value pair to the metadata of image. Case use: organization or image selection.

EXPOSE: Adds port to image metadata. Indicate which port the application within containers will binds to.

ENV: Defines environment variables that are available in container.

ARG: Almost similar to ENV, but it mostly used as build-time variable.

VOLUME: Defines where to store data outside of container. It defines path where podman mounts as persistent volume inside container. Possible to define more than one path to create multiple volumes .

Containerfile Instructions (continue)

```
# This is a comment line 1
FROM registry.redhat.io/ubi8/ubi:8.6 2
LABEL description="This is a custom httpd container image" 3
RUN yum install -y httpd 4
EXPOSE 80 5
ENV LogLevel "info" 6
ADD http://someserver.com/filename.pdf /var/www/html 7
COPY ./src/ /var/www/html/ 8
USER apache 9
ENTRYPOINT ["/usr/sbin/httpd"] 10
CMD ["-D", "FOREGROUND"] 11
```

Container Image Tags

- When building a container image, you can specify an image name to later identify the image.
- The image name is a string composed of letters, numbers, and some special characters.
- An image tag comes after the image name and is delimited by a colon (:). When you omit an image tag, podman uses the default latest tag.
- It is a best practice to specify tags in addition to latest tag. Because latest is the default tag applied to new images, references that use the latest tag can change unintentionally.
- For example, in the full image name **quay.io/jasonwcc/webserver:1.0**
 - a) Registry name: **quay.io**
 - b) Author or Organization name: **jasonwcc**
 - c) Image name: **webserver**
 - d) Tag: **1.0**

The ENV Instruction

- Specify environment dependent configuration, for example, hostnames, ports or usernames.
- Containerized application can use the environment variables at runtime.

To include an environment variable, use the `key=value` format. The following example declares a `DB_HOST` variable with the database hostname.

```
ENV DB_HOST="database.example.com"
```

Then you can retrieve the environment variable in your application. The following example is a Python script that retrieves the `DB_HOST` environment variable.

```
from os import environ

DB_HOST = environ.get('DB_HOST')

# Connect to the database at DB_HOST...
```


The ARG Instruction

Build args are defined by the Dockerfile `ARG` instruction. Once a variable has been defined, later Dockerfile instructions can reference its value:

```
ARG IMAGE_VERSION=3  
FROM alpine:${IMAGE_VERSION}
```

Build argument values are populated at build-time via the `docker build` command `--build-arg` flag:

```
$ docker build --build-arg IMAGE_VERSION=latest -t example:latest .
```

The ARG Instruction - multiple build args

```
ARG foo  
ARG bar
```

Each ARG allows you to pass different variables during the build process.

The `--build-arg` build flag must then be repeated for each argument you want to set:

```
$ docker build --build-arg foo=example --build-arg bar=demo -t example:latest .
```

ARG and ENV Instructions combined

```
ARG VERSION="1.16.8" \  
    BIN_DIR=/usr/local/bin/  
  
ENV VERSION=${VERSION} \  
    BIN_DIR=${BIN_DIR}  
  
RUN curl "https://dl.example.io/${VERSION}/example-linux-amd64" \  
    -o ${BIN_DIR}/example
```

In the preceding example, the ENV instruction uses the value of the ARG instruction to configure the run-time environment variables.

ARG and ENV Instructions combined

If you do not configure the ARG default values but you must configure the ENV default values, then use the `${VARIABLE: -DEFAULT_VALUE}` syntax, such as:

```
ARG VERSION \
    BIN_DIR

ENV VERSION=${VERSION: -1.16.8} \
    BIN_DIR=${BIN_DIR: -/usr/local/bin/}

RUN curl "https://dl.example.io/${VERSION}/example-linux-amd64" \
    -o ${BIN_DIR}/example
```

If developer runs

```
# podman build --build-arg BIN_DIR=/opt/local/bin -t example:latest .
```

- Results in the build process set `VERSION=1.16.8` and `BIN_DIR=/opt/local/bin`

If developer runs

```
# podman build --build-arg BIN_DIR=/opt/local/bin VERSION=3 -t example:latest .
```

- Results in the build process set `VERSION=3` and `BIN_DIR=/opt/local/bin`

If developer runs

```
# podman build -t example:latest .
```

- Results in the build process set `VERSION=1.16.8` and `BIN_DIR=/usr/local/bin`

The ENTRYPOINT and CMD Instructions

ENTRYPOINT

- The ENTRYPOINT and CMD instructions specify the command to execute when the container starts.
- A valid Containerfile must have at least one of these instructions.
- The ENTRYPOINT instruction defines an executable, or command, that is always part of the container execution. This means that additional arguments are passed to the provided command.

CMD

- Runs command when container is started.
- When combined with ENTRYPOINT, CMD passes as argument.

The VOLUME Instruction

- Use the VOLUME instruction to persistently store data.
- The value is the path where Podman mounts a persistent volume inside of the container.
- The VOLUME instruction accepts more than one path to create multiple volumes.
- If VOLUME instruction not specified during build, then local directory is used.

For example, the following Containerfile creates a volume to store PostgreSQL data.

```
FROM registry.redhat.io/rhel9/postgresql-13:1

VOLUME /var/lib/pgsql/data
```

```
[user@host ~]$ podman inspect VOLUME_ID
[
  {
    "Name": "VOLUME_ID",
    "Driver": "local",
    "Mountpoint": "/home/your-name/.local/share/containers/storage/volumes/
VOLUME_ID/_data",
    ...output omitted...
```

Build a simple image

- cat Containerfile
FROM docker.io/library/alpine
CMD ["echo", "hello", "world"]
- Build image using the Containerfile
podman build -t test .
- Verify image is created
podman images
- Start container from the image and view output generated
podman run --name test-container -d test
podman logs test-container

Push image to registry

- Sign up account with docker.io or quay.io
- Log into registry from podman command
podman login quay.io
Username:
Password:
- Push image to registry
podman build -t quay.io/<username>/myfirstimage .
podman push quay.io/<username>/myfirstimage
- Goto quay.io check on myfirstimage repository

Example of Containerfile – only use ENTRYPOINT

```
FROM registry.access.redhat.com/ubi8/ubi-minimal:8.5
ENTRYPOINT ["echo", "Hello"]
```

Running a container from the previous Containerfile with no arguments prints "Hello".

```
[user@host ~]$ podman run my-image
Hello
```

If you provide the container with the "Red" and "Hat" arguments, then it prints "Hello Red Hat".

```
[user@host ~]$ podman run my-image Red Hat
Hello Red Hat
```

Example of Containerfile – only use CMD

```
FROM registry.access.redhat.com/ubi8/ubi-minimal:8.5
CMD ["echo", "Hello", "Red Hat"]
```

Running a container from the previous Containerfile with no arguments prints `Hello Red Hat`.

```
[user@host ~]$ podman run my-image
Hello Red Hat
```

Running a container with the argument `whoami` overrides the `echo` command.

```
[user@host ~]$ podman run my-image whoami
root
```

Example of Containerfile: both used

```
FROM registry.access.redhat.com/ubi8/ubi-minimal:8.5
ENTRYPOINT ["echo", "Hello"]
CMD ["Red", "Hat"]
```

The previous example prints `Hello Red Hat` when you run the container.

```
[user@host ~]$ podman run my-image
Hello Red Hat
```

If you provide the argument `Podman` to the container, then it prints `Hello Podman`.

```
[user@host ~]$ podman run my-image Podman
Hello Podman
```